

WHITEPAPER APRIL 2026 · V1.0

The Agentic Evidence Suite

A reference architecture for agent systems where execution is verifiable, policy is non-bypassable, and evidence is portable.

VERIFIABLE EXECUTION · NON-BYPASSABLE GOVERNANCE · PORTABLE EVIDENCE

What this paper argues

Agent systems now make consequential decisions — prescriptions of content, routing of capital, selection of treatment options, triage of operations work. The observability stack built for microservices assumes humans will read logs after the fact. That assumption fails the moment a decision is consequential and an auditor shows up.

The Evidence Suite is a six-component reference architecture that treats agent execution as a first-class auditable object. Each action produces a canonicalized, hash-chained, signed receipt. Each governance decision emits its own receipt — including denials. An evaluation harness runs bypass-resistance scenarios continuously. A viewer lets a third party open a bundle and verify it offline, without talking to the producing system.

It is deliberately adjacent to existing standards — OpenTelemetry, SLSA, W3C Verifiable Credentials, MCP, CloudEvents — and explicitly not a replacement for any of them. It fills the gap between what those standards describe (transport, spans, provenance of software supply chains) and what regulated AI actually has to answer for (the claim that *this agent took this action under these policies and the evidence is durable*).

THESIS

Logs are a transport format. Telemetry is a signal. **Evidence is a claim that can be verified without trust in the producer.** Most agent systems conflate all three, and that conflation is where audits fail.

WHAT'S IN THE PAPER

- **Chapter 1 — The Evidence Gap.** Why logs, spans, and SLSA provenance don't cover agent execution.
- **Chapter 2 — Invariants.** The three properties any agent evidence layer must hold, and why any two without the third fails.
- **Chapter 3 — Component Map.** The six repositories, what each owns, and how they compose.
- **Chapter 4 — Standards Mapping.** How the Evidence Suite overlaps and diverges from existing standards.
- **Chapter 5 — What's Next.** Open problems, adoption paths, and the invitation.

Logs are not evidence

"We have logs" is the sentence every AI team says when asked about auditability. It is also, almost always, the sentence that ends the audit badly.

A log line is a transport artifact. It was emitted by a process, written to a file, forwarded by a collector, stored by an aggregator, and is now being shown to a human through a search interface. Between emission and display, the line can be edited, re-ordered, lost, sampled, truncated, or synthesized by a later process. The reader has no cryptographic basis to believe the line they're reading is the line that was emitted.

For microservice debugging, this is fine. The cost of a false log is a confused engineer. For agent systems operating on PHI, financial positions, or clinical recommendations, the cost of a false log — or the *inability to prove a log is not false* — is a regulatory finding.

THREE THINGS LOGS CANNOT DO

1. **Prove completeness.** If an action occurred but the log line was dropped, the log cannot show the gap. An auditor asking "show me every tool call in this session" cannot get a verified answer from a log aggregator.
2. **Prove ordering.** Logs are timestamped by the producer, by the collector, and by the store. None of those timestamps are tamper-evident. Post-hoc reordering is trivial.
3. **Prove authorship.** A log line says it came from a service. It does not cryptographically prove which binary, build, or identity emitted it.

WHY EXISTING STANDARDS DON'T CLOSE THE GAP

OpenTelemetry standardizes the transport and semantic conventions for spans, metrics, and logs. It is excellent at that job. It does not sign spans, chain them, or give a verifier any basis to detect omission. An OTel trace is still a log, just one with better structure.

SLSA gives provenance for software supply chains — what source produced what binary, with what builders, with what materials. It answers "how did this artifact come to exist." It does not answer "what did the artifact do once it was deployed."

W3C Verifiable Credentials give tamper-evident claims issued by some party about some subject. VCs are a brilliant primitive and sit adjacent to receipts. But VCs model discrete, human-scale claims (a diploma, a vaccination). Agents take thousands of actions per session; we need a structure tuned for high-volume execution trails, not a credential per action.

MCP standardizes how models connect to tools. It does not define what evidence of a tool call looks like, whether it can be verified offline, or who signs for the call.

CloudEvents gives events a common envelope. Envelopes travel; they do not prove what was inside them two weeks ago.

THE CLAIM

These standards are not wrong. They are solving different problems. Agent systems need a layer *between* them — one that treats each action as a signed, chained, portable receipt, and each policy decision as its own receipt, and composes them into a bundle that a third party can verify without trusting the producer.

That layer is the Agentic Evidence Suite.

Three properties, mutually dependent

An evidence layer holds three properties or it holds none.

VERIFIABILITY BY CONSTRUCTION

Every action produces a canonicalized, hash-chained, signed receipt at the moment of execution. The structure of the receipt — not a post-hoc audit tool — is what makes tampering, reordering, and omission detectable.

Canonicalization means two serializations of the same receipt hash identically. Hash-chaining means each receipt references the previous receipt's hash; dropping or reordering breaks the chain. Signing means an identifiable party attests, non-repudiably, that the receipt was emitted as recorded.

NON-BYPASSABLE GOVERNANCE

Policy is evaluated in the execution path, not beside it. An agent that can reach a tool without passing through policy evaluation is, by definition, ungoverned for that tool. Deny-by-default is the posture; every allow and every deny emits a *decision receipt* — which means an auditor can see why a denial happened just as clearly as why an allow happened.

The failure mode governance layers slip into is "governance as logging." A policy that runs *in parallel* with execution is a reporter. A policy that runs *in line with* execution is a gate.

PORTABLE EVIDENCE

The evidence layer produces bundles — self-contained, verifiable packages that travel. A regulator, a CTO, an auditor, or a successor system can open a bundle and verify it offline with nothing more than the suite's verification tooling and the issuing public keys. No phone calls to the producing system. No "trust us, we have logs."

Portability is what turns receipts from a private ledger into a public good. An architecture that produces beautiful internal evidence but cannot hand a regulator a bundle has solved an internal problem, not the audit problem.

Verifiability without portability is a private ledger. Portability without non-bypassable governance is a partial story. Governance without verifiability is theater. The three are load-bearing for each other.

WHY ANY TWO WITHOUT THE THIRD FAILS

COMBINATION	WHAT YOU GET	WHAT BREAKS
Verifiable + governed, not portable	A well-instrumented internal system	Cannot hand evidence to a third party. Every audit becomes a live conversation with the engineering team.
Verifiable + portable, not governed	A tamper-evident log	You can prove what the agent did. You cannot prove it was permitted. A regulator asks "under which policy" and the answer is "we have logs."
Governed + portable, not verifiable	A policy framework with a nice dashboard	Claims are not cryptographically grounded. A motivated adversary — or a sloppy engineer — can edit the record and nothing detects it.

Six components, one designed system

The Evidence Suite is a deliberate composition: one spec layer, two governance layers, one evaluation layer, and two handoff layers. Each is a standalone OSS repository that composes with the others by hashing and signing contracts, not by runtime coupling.

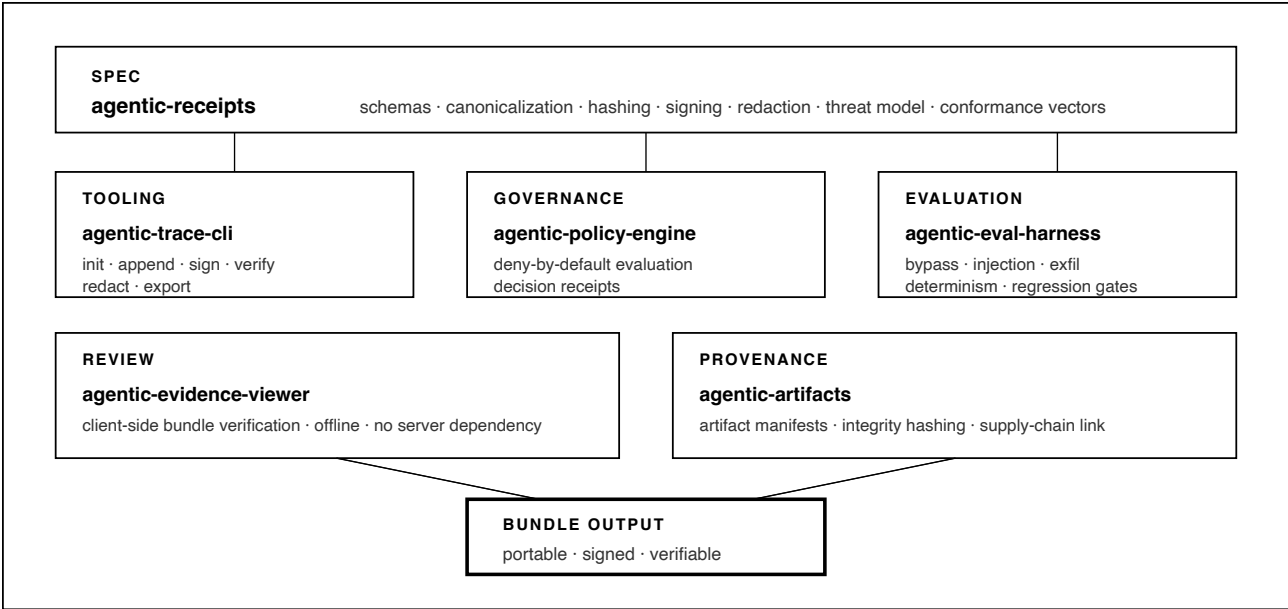


Figure 1 — Evidence Suite component map. Spec at the top defines contracts; four implementation layers emit or verify receipts; bundle output is portable and offline-verifiable.

COMPONENT RESPONSIBILITIES

LAYER	REPOSITORY	OWNS	DOES NOT OWN
Spec	agentic-receipts	Data model, canonicalization, hashing, signing, redaction policy, threat model, conformance vectors	Transport, storage, UI
Tooling	agentic-trace-cli	Local init/append/sign/verify/redact/export over files	Policy evaluation, UI
Governance	agentic-policy-engine	Deny-by-default evaluation at enforcement points; emits decision receipts for allows and denies	Receipt schema (consumes), storage

LAYER	REPOSITORY	OWNS	DOES NOT OWN
Evaluation	agentic-eval-harness	Red-team scenarios (bypass, injection, PHI exfil, budget overrun, determinism) and regression gates	Runtime governance (uses)
Review	agentic-evidence-viewer	Offline, client-side bundle verification	Authoring, signing, storage
Provenance	agentic-artifacts	Artifact manifests with integrity hashing; linkage to SLSA-style supply chain	Runtime execution

Adjacent, not redundant

The Evidence Suite is not a replacement for OpenTelemetry, SLSA, W3C VC, MCP, or CloudEvents. It is a layer above them, tuned to the specific problem of high-volume agent execution under regulatory scrutiny.

STANDARD	OVERLAP	DIVERGENCE	RELATIONSHIP
OpenTelemetry	Both model execution as structured records. Span IDs and receipt IDs are both references to bounded units of work.	OTel spans are not signed, not chained, and omissions are undetectable. Receipts are signed, hash-chained, canonicalized.	OTel carries receipts. A receipt can be emitted as a span attribute or <i>alongside</i> a span. OTel is the transport; receipts are the evidence payload.
SLSA	Both care about tamper-evidence and signing.	SLSA describes how artifacts come to exist. Receipts describe what agents do at runtime.	Complementary. agentic-artifacts links receipts to SLSA-style provenance — every artifact a receipt references has a verifiable build lineage.
W3C Verifiable Credentials	Both are cryptographically verifiable claims issued by an identified party.	VCs target discrete, human-scale claims. Receipts target high-volume execution trails with hash-chaining.	Receipts can be framed as VCs for specific use cases (e.g., an "authorized action" credential). Day-to-day, hash-chained receipts are the right primitive for execution volume.
MCP	Both concern agent–tool interactions.	MCP defines how tools are called. Receipts define what evidence of the call looks like.	A signed receipt can be emitted per MCP tool call, making the suite a natural evidence layer for MCP-based agent platforms.
CloudEvents	Both add structure to event records.	CloudEvents is an envelope. Receipts are the claim inside the envelope.	A receipt can ride a CloudEvents envelope across system boundaries without losing its signature.

DESIGN STANCE

Receipts carry what existing standards don't try to carry: **a durable, verifiable claim about a specific agent action, its inputs, its policy decision, and its outputs, signed by an identified party, chained into an ordered record, and packaged into a bundle a third party can verify offline.** Everything else — transport, provenance, credentials, envelopes — is delegated to the standards already doing those jobs well.

INTEROP POSTURE

The Evidence Suite will move toward W3C / IETF working groups as the spec stabilizes. The goal is not to compete with standards but to contribute a missing layer. Conformance vectors ship today so implementations can align before any standardization.

Open problems and the invitation

The Evidence Suite is a v1 reference architecture. Several problems remain genuinely open. A mature community around this layer — not any single implementer — is what closes them.

OPEN PROBLEMS

- **Post-quantum signature path.** Ed25519 is today's default. A dual-signature scheme (classical + Dilithium3 or similar) needs a migration plan that doesn't break existing chains.
- **Cross-organization bundle interop.** When Agent A (Organization X) calls Agent B (Organization Y), whose chain does the receipt belong to? A shared-custody model is needed.
- **Redaction under verification.** Redaction today preserves hash integrity but loses content. Selective-disclosure cryptography (BBS+ signatures, zkSNARKs in limited domains) is promising but not yet practical at execution volumes.
- **Receipt volume and archival.** A high-volume agent emits tens of thousands of receipts per session. Cold-storage and retrieval patterns that preserve verifiability at archive scale are an open operational question.
- **Regulatory alignment.** FDA, EMA, and equivalent agencies are publishing guidance for AI/ML systems. Mapping Evidence Suite outputs to specific regulatory control families (21 CFR Part 11, Annex 11, GxP validation expectations) is work in progress.

ADOPTION PATHS

1. **Single-team pilot.** One agentic workflow, receipts emitted in parallel with existing logs, viewer used for review. Low risk, builds internal familiarity. Typical timeline: 4–8 weeks.
2. **Platform-level adoption.** Policy engine placed at the enforcement point for all agent tool calls; receipts become the primary audit record. Typical timeline: 3–6 months.
3. **Cross-team standardization.** Receipt schema adopted across multiple agent systems in an organization; bundles become the exchange format for cross-system collaboration. Typical timeline: 6–12 months.
4. **Ecosystem contribution.** Conformance vectors adopted by vendors; cross-implementation bundles verify. Timeline: ongoing, community-dependent.

HOW THIS TIES TO ATVC

The Evidence Suite is the technical substrate. ATVC — the Agentic Trust Validation Certification framework — is the methodology for bringing the substrate to bear on a real system: 100 steps across

Ontology → Architecture → Engineering → Enablement, with 211 validation documents. Organizations that want the Evidence Suite implemented in a regulated context typically run ATVC to get there.

THE INVITATION

The Evidence Suite is open-source and being actively developed. The fastest ways to engage:

- Read the specs at github.com/cmangun/agentic-receipts.
- Run the conformance vectors against any implementation you maintain.
- Open issues, propose changes, or contribute ADRs.
- If your organization has a regulated AI program and wants architecture-level engagement, see the back cover.

ABOUT THE AUTHOR

Christopher Mangun is an Agentic Systems Architect based in Brooklyn, NY. He designs AI systems where execution is verifiable, policy is non-bypassable, and evidence is portable. His practice focuses on regulated industries — healthcare, pharmaceutical, medical devices — where the gap between "shipped" and "auditable" is the hardest part of the problem. He is the author of ATVC and the designer of the Agentic Evidence Suite.

HOW TO ENGAGE

EMAIL	cmangun@gmail.com
SITE	healthcare-ai-consultant.com
GITHUB	github.com/cmangun
LINKEDIN	linkedin.com/in/christophermangun
ATVC	enterprise-ai-playbook-demo.vercel.app · access code: ATVC

This whitepaper is version 1.0, April 2026. It is published under a permissive license for redistribution with attribution. The Agentic Evidence Suite specifications, ATVC framework, and associated templates are trademarks and copyrighted works of Christopher Mangun. Referenced standards are the property of their respective organizations.